

Personal Loan Ledger Mobile Application

System Architectural Design

Shared Loan Tracking, Reminders, Repayment Confirmation, Dashboard Analytics, and Bank Profile Sharing

Document	System Architectural Design
Version	Version 1.0 - Product Foundation
Date	September 2026
Purpose	Defines the system context, component boundaries, trust zones, deployment topology, scaling model, and architecture decisions for the React Native loan ledger.

Product scope note: The application tracks peer-to-peer loans and repayment acknowledgements. It does not custody funds, execute bank transfers, provide credit scoring, or automatically enforce legal debt collection in the initial release.

Table of Contents

1. Architecture Executive Overview
2. Architectural Drivers
3. System Context
4. Recommended Architecture Style
5. Logical Component Architecture
6. Mobile Client Architecture
7. Backend Modular Boundaries
8. Data Architecture
9. Communication and API Architecture
10. Identity and Trust Boundaries
11. Loan State and Workflow Architecture
12. Notification/Event Architecture
13. Dashboard Read Architecture
14. Bank Profile Security Architecture
15. Deployment Architecture
16. Scalability Model
17. Availability and Resilience
18. Performance Architecture
19. Security Architecture
20. Privacy and Data Lifecycle
21. Observability Architecture
22. Disaster Recovery Architecture
23. Architecture Decision Records
24. Alternative Architectures Considered
25. Evolution Path and Future Services
26. Architecture Review Checklist

1. Architecture Executive Overview

The recommended first-release architecture is a React Native mobile client backed by a modular monolith API, PostgreSQL transactional storage, a background job subsystem, push-notification provider integration, and optional object storage. This balances correctness, speed of delivery, and future scalability.

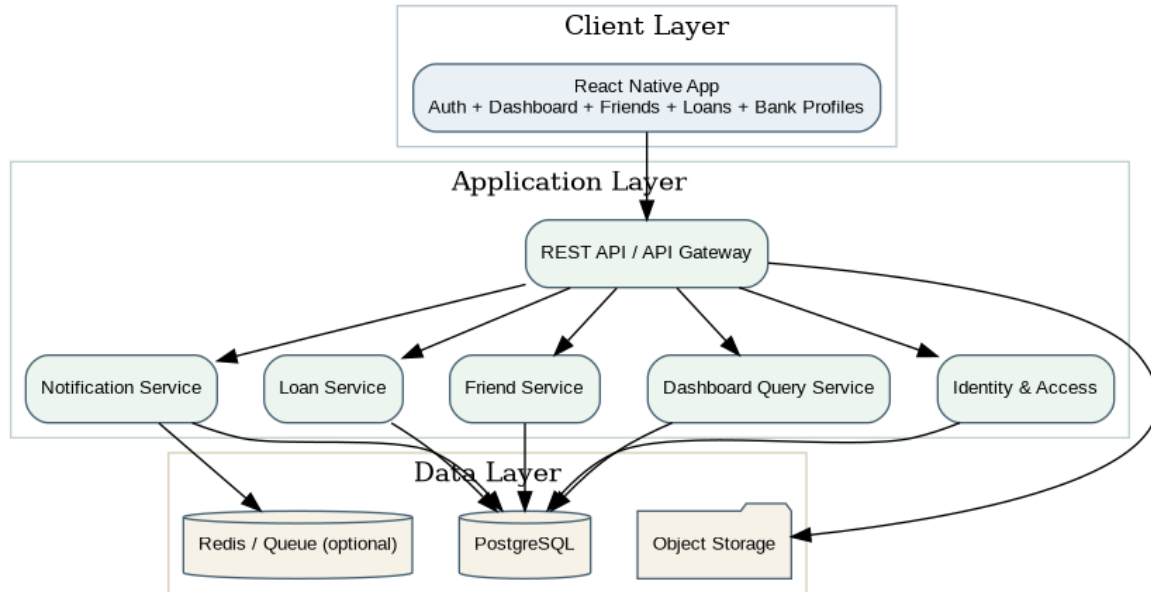


Figure 1. Logical component architecture.

- Financial truth lives on the server, not the device.
- All critical state transitions are transactional and auditable.
- The architecture separates write workflows from dashboard/read concerns without prematurely creating microservices.
- Sensitive bank data sits behind a tighter authorization and encryption boundary than ordinary profile data.

2. Architectural Drivers

Driver	Architectural Impact
Shared truth between two users	Central authoritative backend and immutable event history.
Financial correctness	Relational transactions, constraints, decimal money types, explicit state machine.
Reminder timing	Background jobs and reconciliation scheduler.
Multi-currency	Currency-scoped aggregates and no implicit conversion.
Sensitive bank details	Field-level encryption, explicit share objects, masked-by-default APIs.
Mobile connectivity	Online-first mutations, idempotency, resumable reads.
Rapid MVP delivery	Modular monolith over microservices.
Growth	Stateless API, horizontal scale, clear extraction boundaries.

3. System Context

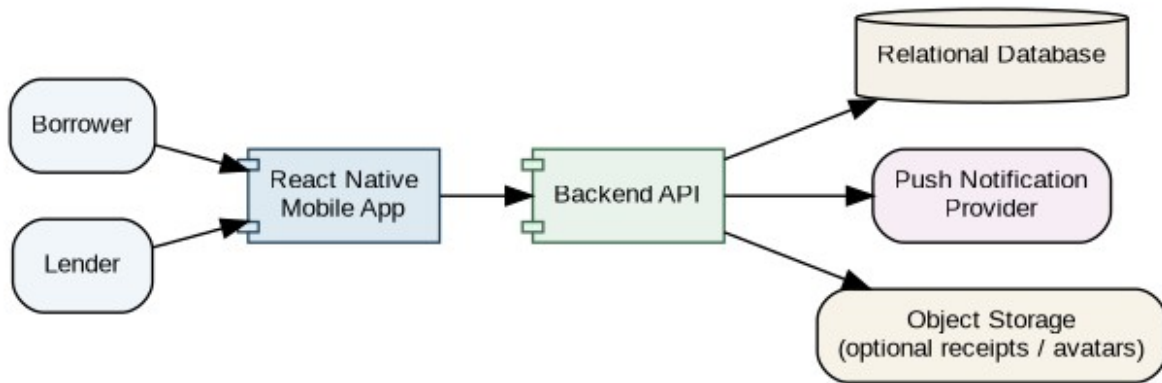


Figure 2. External context and primary platform dependencies.

- Actors: borrower, lender, support/admin.
- External systems: push provider, email/SMS verification provider if used, object storage, monitoring/analytics platform.
- No bank/payment rail is required in the initial architecture; bank profiles are informational data only.

4. Recommended Architecture Style

- Use a modular monolith: one deployable backend with strongly separated modules and database ownership conventions.
- Benefits: local transactions across loans/repayments, simpler testing, fewer network failure modes, lower operational burden.
- Avoid a “big ball of mud” by enforcing dependency direction and module public interfaces.
- Extract a module into a service only when scale, isolation, organizational ownership, or independent release cadence creates measurable value.

Rule of thumb: Do not introduce microservices merely because the domain has several nouns. Correctness and delivery speed matter more than service count for this product stage.

5. Logical Component Architecture

Component	Owns	May Depend On
Identity	Users, credentials, sessions	Notification for verification
Friendships	Connections, blocks	Identity
Loans	Loan lifecycle, terms, balance semantics	Identity, Friendships, Audit, Notifications
Repayments	Repayment lifecycle	Loans, Audit, Notifications
Bank Profiles	Sensitive payment destinations/shares	Identity, Loans, Audit
Dashboard	Aggregated read models	Loans, Repayments, Friendships
Notifications	In-app records, push delivery	Identity, Jobs
Jobs	Delayed/retry work	Notifications, Loans
Audit	Append-only events	Identity; called by domain modules
Admin	Support resolutions	Domain modules through privileged interfaces

6. Mobile Client Architecture

- Presentation layer: screens/components, accessibility, navigation, formatting.
- Feature/domain layer: typed models, form schemas, loan status presentation, pure money/date helpers.
- Server-state layer: query/mutation hooks with cache invalidation and request deduplication.
- Infrastructure layer: HTTP client, secure storage, push registration, analytics, file upload.
- Local state is limited to UI/session preferences; authoritative loan state comes from server queries.
- Deep links target stable resources such as loan ID but never expose bank identifiers or tokens.

7. Backend Modular Boundaries

- Each backend module has controller/API boundary, application service, domain rules, repositories, and tests.
- Cross-module access should use service interfaces, not arbitrary direct table writes.
- Loans module owns loan status. Repayments module requests loan state transition through Loans service rather than directly updating status.
- Audit creation is invoked transactionally with business state changes.
- Notifications are side effects that may be persisted transactionally then delivered asynchronously.

8. Data Architecture

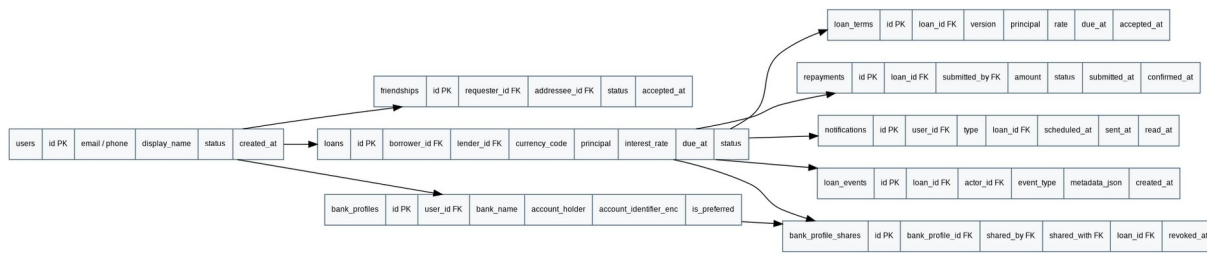


Figure 3. Simplified entity relationship view. Detailed schema appears in the Database Design document.

- PostgreSQL is the system of record.
- Normalized tables hold users, friendships, loans, terms, repayments, bank profiles/shares, notifications, and events.
- JSON is used only for extensible safe metadata, not to avoid relational modeling of core fields.
- Money uses NUMERIC/DECIMAL; all identifiers use UUID or equivalent globally unique IDs.
- Database constraints encode invariants wherever possible.

9. Communication and API Architecture

- REST/JSON is recommended for MVP due to simple mobile interaction patterns and excellent tooling.
- Version API at /api/v1 and use OpenAPI as the machine-readable contract.
- All timestamps UTC in transport; client renders in user timezone.
- Idempotency keys protect replay-prone POST actions such as loan acceptance and repayment submission.
- Pagination and filtering are server-side to avoid exposing entire histories.
- Background jobs are asynchronous internal communication; the user-facing response should indicate committed business state even if push delivery happens later.

10. Identity and Trust Boundaries

- Trust zones: mobile device (untrusted), public API boundary, application network, data stores, operational/admin plane.
- Never authorize based on UI role or client-supplied actor ID.
- Admin interfaces require stronger authentication and separate permissions from ordinary user API.
- Sensitive bank-decryption operation is isolated behind explicit authorization and audit.
- Support tooling should not expose secrets by default.

11. Loan State and Workflow Architecture



Figure 4. Loan lifecycle interaction at a high level.

- The loan state machine is explicit; invalid transitions are rejected with conflict errors.
- Material term changes are versioned rather than overwritten.
- Repayment is modeled as a separate entity so a claim can exist without prematurely completing the loan.
- Overdue is a status/projection based on due date and outstanding balance, not a destructive mutation of terms.

12. Notification/Event Architecture

- Use domain events internally as a design concept even if no external event bus exists initially.
- Business transaction writes an outbox/notification intent or directly persists in-app notification; asynchronous worker handles push.
- Delayed reminders are scheduled from due date and periodically reconciled.
- At-least-once delivery is expected; consumers/processors are idempotent.
- Eventual consistency is acceptable for push delivery and some dashboard caches, but not for loan acceptance or repayment confirmation.

13. Dashboard Read Architecture

- Dashboard is a read-focused API that composes data from loans/repayments and friendship summaries.
- Initially use indexed SQL aggregates for correctness and simplicity.
- If load grows, create projection tables or materialized views updated from domain events/outbox.
- Every metric drill-down must be reproducible from source records so cached totals can be reconciled.

14. Bank Profile Security Architecture

- Treat bank account identifiers as higher sensitivity than ordinary loan metadata.
- Encrypt account identifier fields with KMS-backed envelope encryption or equivalent.
- Store masked derivative (e.g., last four) separately for display to avoid decrypting list views.
- Authorization is based on owner or active bank_profile_share scoped to recipient and optional loan.
- Reveal endpoint can require recent authentication and should be rate-limited/audited.
- Do not transmit sensitive identifiers in analytics, logs, URLs, crash reports, or push payloads.

15. Deployment Architecture

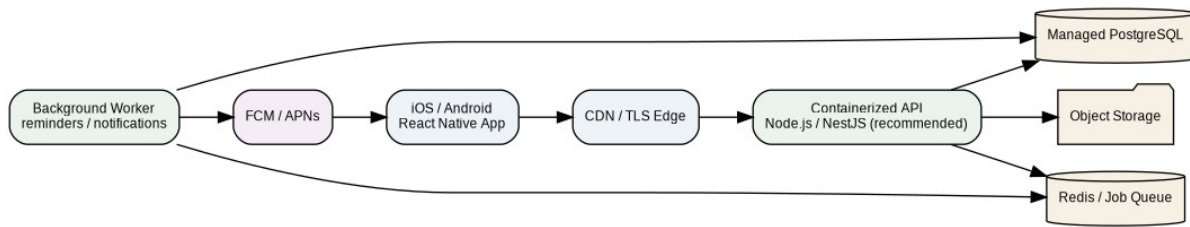


Figure 5. Production deployment reference topology.

- Mobile app connects through TLS edge/load balancer to stateless API containers.
- Workers run independently and consume a durable job queue for delayed reminders/retries.
- Managed PostgreSQL is private-networked where possible.
- Object storage remains private; signed URLs provide limited access.
- Monitoring/telemetry exporters send sanitized data to observability tooling.

16. Scalability Model

Scale Dimension	Response
More API traffic	Add stateless API instances.
More background reminders	Add worker concurrency/instances and partition queues if needed.
Read-heavy dashboards	Optimize indexes -> cache -> projection table/materialized view.
Large history	Cursor pagination and archival/partition strategy if necessary.
Database growth	Tune queries/indexes, vertical scaling, read replicas for non-transactional reads, partition high-volume event tables later.
Global users	Use regional edge/CDN; keep single write region initially unless multi-region write requirements justify complexity.

17. Availability and Resilience

- API instances are replaceable; no session memory dependency.
- Queue/provider outage does not block authoritative loan state; push can retry later.
- Reminder reconciliation repairs missing jobs from source data.
- Database is a single critical dependency and receives managed HA/backups appropriate to production tier.
- Circuit breakers/timeouts apply to external push/email/SMS providers.
- Client surfaces graceful retry and authoritative refresh after transient errors.

18. Performance Architecture

- Budget latency primarily for database queries and network; avoid chatty mobile API patterns.
- Use aggregated dashboard endpoint instead of many per-card calls.
- Loan list returns compact summary DTO; detail/timeline fetched on demand.
- Friend list summary avoids per-friend subqueries through grouped aggregation.
- Push registration and analytics are non-blocking relative to critical screens.

19. Security Architecture

Threat	Architectural Mitigation
IDOR / unauthorized loan access	Relationship-aware authorization on server.
Account takeover	Strong auth, session revocation, rate limits, recent-auth for sensitive reveal.
Data leak through logs	Central redaction policy and structured safe logging.
Bank identifier breach	Field encryption + least privilege + masked default.
Duplicate financial actions	Idempotency + transactions + state guards.
Tampered client	Server recomputes totals and ignores client ownership claims.
Malicious attachments	Private storage, type/size validation, scanning if attachments enabled.
Admin abuse	RBAC, stronger auth, reason-required audited interventions.

20. Privacy and Data Lifecycle

- Classify data: public-ish profile display fields, private contact identifiers, confidential loan data, highly sensitive bank identifiers, security credentials.
- Minimize collection. Do not require government identity unless product strategy changes.
- Account deletion separates personal profile erasure from retention of shared loan records where legitimate/legal retention applies.
- Bank shares can be revoked; encrypted bank profiles can be deleted/archived subject to active-loan policy.
- Backups inherit retention and deletion-lag documentation.

21. Observability Architecture

- Correlation ID travels from edge through API and job logs.
- Business event IDs allow support to trace a loan transition without exposing bank data.
- Metrics separate technical failure from user validation conflict.
- Dashboards cover API health, DB health, worker health, push health, and integrity/reconciliation.
- Alerts are actionable and tied to runbooks, not merely noisy threshold notifications.

22. Disaster Recovery Architecture

- Automated backups and point-in-time recovery where supported.
- Document RPO/RTO and test restore into isolated environment at scheduled intervals.
- Object storage enables versioning/lifecycle as appropriate.
- Infrastructure configuration is reproducible through IaC when production maturity warrants.
- Disaster recovery prioritizes authoritative loan/event data over caches or derived projections, which can be rebuilt.

23. Architecture Decision Records

ADR	Decision	Reason
ADR-001	React Native for mobile	Single codebase for iOS/Android with native access for push/secure storage.
ADR-002	Modular monolith backend	Lower operational complexity; strong transaction support.
ADR-003	PostgreSQL as system of record	Relational integrity and robust transactions.
ADR-004	REST API	Simple mobile contract and broad tooling.
ADR-005	Flat per-loan interest in MVP	Unambiguous calculation; APR deferred.
ADR-006	Currency-separated dashboard	Prevents misleading totals without FX rates.
ADR-007	Bank details informational only	Avoid payment-processing scope in MVP.
ADR-008	Append-only loan event history	Traceability and dispute support.
ADR-009	Online-first financial mutations	Avoid complex unsafe offline conflict resolution.

24. Alternative Architectures Considered

Alternative	Why Not First Choice
Microservices from day one	Operational overhead, distributed transactions, duplicated infrastructure, slower delivery.
Serverless-only backend	Can work, but delayed jobs/transactions/connection management may require additional platform-specific complexity; still viable if team expertise is strong.
Firebase/NoSQL as sole source	Possible, but relational multi-entity integrity, reporting, and transaction/query patterns favor PostgreSQL for this domain.
Local-first/offline financial writes	Conflict resolution is difficult for two-party approvals and can undermine shared-truth semantics.
Direct bank API integration	Adds KYC/payment/reconciliation/compliance scope unrelated to validating core product demand.

25. Evolution Path and Future Services

- Phase A: modular monolith + PostgreSQL + single worker/queue.
- Phase B: introduce dashboard projection table and dedicated notification workers as load grows.
- Phase C: extract notification service if provider integrations and throughput justify independent deployment.
- Phase D: extract analytics/read service or event stream if reporting becomes significant.
- Phase E: payment service only after product/legal decision to support transfers; keep bank/payment compliance boundary isolated.

26. Architecture Review Checklist

- Are loan and repayment transitions transactional and idempotent?
- Can any user access another user's loan or bank details by changing an ID?
- Are dashboard metrics explainable and currency-separated?
- Can lost reminder jobs be reconstructed?
- Are term modifications versioned?
- Can support repair data without impersonating users or deleting history?
- Are secrets and sensitive values excluded from logs and push payloads?
- Can the database be restored from backup and derived projections rebuilt?
- Does the design avoid unnecessary microservices while preserving module boundaries?
- Are future payment integrations separable from the core shared-ledger system?